

ProtejaJA — Documentação para Explicar (formato fácil, passo a passo)

Objetivo deste documento: você conseguir **explicar** o projeto na apresentação, entendendo cada parte sem se perder. Vá com calma, uma seção por vez. Cada seção tem um quadro **"O QUE DIZER"** com a frase pronta para a banca.

1. VISÃO GERAL — O que é o ProtejaJA?

ProtejaJA é um site de **denúncias**. A pessoa cria uma conta, entra, e faz uma denúncia (ex.: comprar produto vencido no Hipermercado e não quererem trocar).

- **Quem denuncia** → qualquer usuário cadastrado.
- **Quem é denunciado** → estabelecimento / empresa / pessoa (ex.: **Hipermercado**).
- **Prova** → o usuário pode anexar imagem, PDF ou vídeo.
- **Exclusão** → o usuário pode apagar a conta definitivamente (dados, denúncias e provas).

STACK (tecnologias) — salve isso, o professor vai perguntar: | Parte | Ferramenta | Versão | Para que serve | |---|---|---|---| | Tela (front-end) | HTML + Tailwind CSS + CSS | Tailwind via CDN | Aparência rosa, botões, centralizar | | Servidor (back-end) | **Python + Flask** | Python 3.14 / Flask 3.1 | Lógica, validações, rotas | | Banco de dados | **SQLite** | nativo | Guardar usuários e denúncias | | Container | **Docker** | Docker + Compose | Rodar em qualquer máquina |

QUE DIZER (tecnologias):

"O front-end é HTML com Tailwind CSS, o back-end é Python com Flask, o banco é SQLite e o Docker empacota tudo. As senhas são criptografadas com o módulo Werkzeug do Flask."

QUE DIZER:

"O ProtejaJA é um site de denúncias. O usuário se cadastra, faz login e denuncia alguém ou alguma empresa, podendo anexar uma prova. O sistema bloqueia ofensas e guarda a senha de forma criptografada."

1.5 FRONT-END, BACK-END E BANCO DE DADOS (o que é cada um — perguntinha certa do professor)

Pense no site como um **restaurante**:

Parte	No restaurante	No ProtejaJA	Ferramenta
Front-end	O salão, o cardápio e o garçom (o que você vê e toca)	As páginas que abrem no navegador: Home rosa, botões, formulários	HTML + CSS + Tailwind
Back-end	A cozinha (onde o pedido é preparado com as regras)	O servidor Flask : recebe o formulário, valida, aplica regras e responde	Python + Flask
Banco de dados	A despensa/caderno (onde a receita é anotada e guardada)	O arquivo SQLite que guarda usuários e denúncias	SQLite

Como eles conversam (a ordem): Você preenche a tela (front-end) ↓ envia o formulário Flask processa (back-end) ↓ grava/busca SQLite guarda os dados (banco) ↑ devolve A tela mostra o resultado (front-end)

Onde fica cada código (mostre no VS Code): | Parte | Arquivo | |---|---| | Front-end | backend/templates/*.html + home/css/styles.css | | Back-end | backend/app.py, backend/database.py, backend/profanity.py | | Banco de dados | db/app.db (gerado pelo database.py) |

QUE DIZER (front-end, back-end, banco):

"O front-end é o que aparece no navegador — as telas rosas feitas com HTML e Tailwind. O back-end é o servidor Flask, que recebe o que o usuário digita, valida e aplica as regras — é ele quem trata a senha e o filtro antiofensa. E o banco de dados SQLite guarda tudo: os usuários e as denúncias."

Truque para não confundir na hora: - **Front-end** = "o que o usuário VÊ" (tela). - **Back-end** = "o que acontece POR TRÁS" (validação e regras). - **Banco** = "onde fica GUARDADO" (os dados).

2. O CAMINHO DOS DADOS (como as 3 partes se falam)

1) Você digita na TELA 2) A TELA envia para o SERVIDOR (Flask) 3) O SERVIDOR valida e grava no BANCO (SQLite) 4) O SERVIDOR devolve a resposta para a TELA

QUE DIZER:

"O fluxo é: o usuário preenche o formulário, o Flask recebe, valida os dados, grava no SQLite e responde com a página. É um serviço cliente-servidor com banco de dados."

3. AS TELAS (uma por uma)

3.1 Home (página inicial)

- Título grande **ProtejaJA**, cor rosa, letras pretas.
- Botões: **Criar conta** e **Entrar**.

3.2 Cadastro

- Campos: **nome, sobrenome (separados e obrigatórios), e-mail, data de nascimento, cidade, senha, endereço completo**.
- Exige **nome E sobrenome preenchidos** — não deixa criar conta sem os dois.
- Senha transformada em **hash** (criptografia) antes de gravar.
- Depois de salvar → **vai para a tela de login**.

3.3 Login

- E-mail + senha.
- Link visível: **"Recuperar senha"**.

3.4 Recuperar senha

- Pede e-mail + data de nascimento.
- Se bater, gera **senha temporária** e mostra na tela.

3.5 Denúncias (a tela mais importante)

O formulário tem **5 campos** (a banca vai adorar isso):

Campo	O que é	Exemplo
Quem você está denunciando?	O estabelecimento/empresa	Hipermercado
Sobre o quê?	O problema	Produto vencido
Categoria	Tipo	Produto / Local / Serviço
Descrição	Detalhe do acontecido	Comprei vencido e não trocam
Prova (anexo)	Imagem/PDF/vídeo	Foto do produto

Depois de enviada, aparece em **"Minhas denúncias"** com quem, o quê e a foto. Clicando em **"■ Ver detalhes"**, abre a página completa da denúncia (descrição inteira + botão para abrir a prova anexada).

3.6 Excluir conta (novo)

- No menu, o ícone **■** ao lado do nome.
- Pede a **senha** para confirmar.
- Apaga **de verdade**: cadastro, todas as denúncias e as provas do disco.
- Depois da exclusão, o e-mail não consegue mais entrar (testado!).

3.7 Painel do administrador (moderação)

- Conta especial: **admin@protejaja.com / admin123** (criada automaticamente).
- No menu, o link **■ Painel admin** (só aparece para admin).
- O admin vê **todas** as denúncias com status e escolhe **■ Aprovar** ou **■ Rejeitar**.

3.8 Ver denúncias (página pública)

- Link **"Ver denúncias"** no menu (funciona sem login).
- Mostra **apenas as denúncias aprovadas** pelo admin.
- É o ciclo: usuário denuncia → admin aprova → comunidade vê.

QUE DIZER (do painel admin):

"Existe uma conta de administrador que modera as denúncias: ele aprova ou rejeita cada uma. Só as aprovadas aparecem na página pública para toda a comunidade ver — assim não vai denúncia ofensiva ou falsa para o ar."

QUE DIZER (da tela de denúncia):

"Aqui o usuário informa quem está sendo denunciado e o que aconteceu. Por exemplo: comprou um produto vencido no Hipermercado e não quiseram trocar. Ele digita Hipermercado, anexa a foto do produto vencido e envia."

4. A SEGURANÇA (5 pontos que a banca adora)

1. **Senha criptografada** — o banco guarda só um hash, nunca a senha.
2. **Filtro antiofensa** — bloqueia palavras ofensivas no nome, endereço, descrição, assunto e em "quem está denunciando".
3. **Sem SQL Injection** — as consultas usam ? (parâmetros), nunca texto colado.
4. **Login obrigatório** — só dá para denunciar logado; cada usuário só vê as próprias denúncias.
5. **Exclusão definitiva** — apagar a conta exige a senha e remove os arquivos do disco, sem deixar vestígio.

QUE DIZER:

"Senhas nunca ficam em texto puro: viram um hash. O texto digitado passa pelo filtro antiofensa, que bloqueia xingamentos. As consultas ao banco usam parâmetros, evitando SQL Injection, e só usuários logados podem denunciar. E quem pede a exclusão da conta tem os dados e as provas removidos definitivamente — testamos e o e-mail não consegue mais entrar."

5. O BANCO DE DADOS (2 tabelas)

Tabela	Guarda	Campos principais
users	Usuários	nome, email, data_nascimento, cidade, endereco, senha_hash
reports	Denúncias	denunciado, assunto , categoria, descricao, anexo
• Cada denúncia tem user_id que liga ao dono da conta (chave estrangeira). • O banco fica no arquivo db/app.db.		

QUE DIZER:

"São duas tabelas: users e reports. A denúncia guarda quem foi denunciado (denunciado), sobre o que (assunto), a descrição e o anexo, além do usuário que fez a denúncia."

6. OS ARQUIVOS (o que cada um faz, sem decoreba)

Arquivo	Papel	O que ele faz no meu projeto
run.py	Liga o servidor	Inicia o Flask na porta 5000
backend/app.py	Cérebro	Todas as rotas: cadastro, login, recuperar, denúncias, detalhes, excluir conta
backend/database.py	Banco	Cria tabelas, salva e busca usuários/denúncias, apaga conta completa
backend/profanity.py	Filtro	Lista de ofensas + normalização (pega até "c@r@lho")
backend/templates/	Telas	HTML de cada página (denuncias, ver_denuncia, excluir_conta...)
home/	Visual	CSS rosa e o JS da home
Dockerfile	Container	Empacota o projeto para rodar em qualquer máquina

Sugestão para a banca: abra só 2 arquivos e aponte: - app.py → onde a senha é criptografada (generate_password_hash) e a exclusão (excluir_conta). - profanity.py → a lista de palavras bloqueadas.

7. DOCKER (se perguntarem "como roda em qualquer lugar?")

- **Dockerfile** → a "receita" da imagem (Python + código).
- **docker-compose.yml** → sobe o site na porta 5000 e preserva banco + uploads.

Rode com: powershell docker compose up -d --build

QUE DIZER:

"Com o Docker, qualquer pessoa roda o projeto com um único comando, sem precisar instalar nada manualmente. Os dados ficam em volumes, então não se perdem quando o container reinicia."

8. PERGUNTAS IMPORTANTES DO PROFESSOR (LGPD, dados reais e criptografia)

8.1 Tem segurança sobre a lei LGPD?

O que o projeto JÁ garante (princípios da LGPD aplicados):

Princípio da LGPD	No ProtejaJA
Privacidade desde a concepção	■ Coleta apenas o necessário (cadastro enxuto, sem dados a mais)
Segurança dos dados	■ Senha criptografada + login obrigatório
Direito de excluir os dados	■ Botão ■■ apaga cadastro, denúncias e provas
Controle (cada um vê só o seu)	■ Cada usuário vê apenas as próprias denúncias
Termo de consentimento / Política de Privacidade	■■ Não existe (seria exigido em empresa real)
Encarregado de dados (DPO), registro de vazamento	■■ Só obrigatório para empresa real, não para projeto escolar

QUE DIZER:

"Em um projeto integrador não há empresa real, então não precisamos do aparato completo da LGPD, como o DPO e o termo de consentimento. Mas aplicamos os princípios dela: coletamos só o necessário, criptografamos a senha e garantimos o direito à exclusão — o usuário apaga a conta e todos os dados. Na vida real, para colocar o site no ar, acrescentaríamos o termo de consentimento e a política de privacidade."

Frase curta (se ele cobrar):

"Sigo o espírito da LGPD: coleta mínima, segurança e direito de apagar os dados — e o botão de excluir conta já funciona."

8.2 Como garantir que os dados sejam reais?

Verdade técnica: não dá para provar que uma pessoa é real só com um formulário. Qualquer um pode digitar "Teste Teste" e um e-mail que existe.

Método de verificação	Nível	No ProtejaJA
Confirmação por e-mail (link de verificação)	Médio	■■ Não (posso implementar)
Validação de CPF (checagem matemática dos dígitos)	Médio	■■ Não (posso implementar)
Integração com governo (validação facial/central.gov.br)	Alto	■ Fora do escopo escolar

QUE DIZER:

"Garantir que uma pessoa é real exige verificação externa, como confirmação por e-mail ou validação de CPF. No projeto integrador, o cadastro é autodeclarado, como na maioria dos sites simples. Se o escopo permitisse, o próximo passo seria a confirmação por e-mail."

8.3 Criptografia dos dados

Dado	Como está protegido
Senha	■ Vira um hash com scrypt (Werkzeug do Flask) — nunca fica em texto puro
Conexão (transporte)	■■ Site roda em <code>localhost</code> ; o HTTPS existe quando vai para a internet
Provas/anexos	■ Ficam no disco do servidor, acessíveis só com login

QUE DIZER:

"As senhas nunca são armazenadas em texto puro: usamos **scrypt**, que transforma a senha em um hash irreversível. Se o banco vazar, a senha não vaza — só o hash, que não dá para transformar de volta em senha."

Truque da bateria (para não confundir): - **Hash** = senha virada em "embaralhado" que não volta (é só um sentido). - **HTTPS** = protege o caminho até o servidor (só quando o site está na internet).

9. PLANO DA APRESENTAÇÃO (7 passos, com o que falar)

1. **Abrir** a Home → "Este é o ProtejaJA, site de denúncias."
2. **Cadastrar** um usuário → "Preencho nome, data de nascimento, cidade e endereço." (mostrar que foi para o login)
3. **Entrar** → "E-mail e senha; aqui tem o link de recuperar senha."
4. **Denunciar** → "Vou denunciar o Hipermercado por produto vencido" → preencher os campos + anexar foto → enviar.
5. **Mostrar a denúncia** listada com "Hipermercado" e a foto → clicar em "■ **Ver detalhes**" e mostrar a página de detalhes.
6. **Segurança** → "Senha fica só como hash; e as ofensas são bloqueadas" (se possível, digite um xingamento e mostre o bloqueio).
7. **Excluir conta** → "E se o usuário quiser sair de vez, apaga a conta: cadastro, denúncias e provas — sem deixar vestígio." (opcional, se sobrar tempo)

10. COMO RODAR (se precisar demonstrar)

```
powershell docker compose up -d --build
Abrir: http://127.0.0.1:5000/home
```

11. DICAS PARA QUEM TEM TDAH (para você)

- Explique a **ordem da história**: cadastro → login → denúncia → segurança.
- Não decore código: decore os **4 cartões** (o que é, como fala com o banco, as telas, a segurança).
- Se esquecer uma palavra, está tudo aqui em "**QUE DIZER**".
- Treine 2 vezes na frente do espelho/imprimindo este PDF e lendo em voz alta.